RelaLeap WBIC/SGLD/LLC Estimator Implementation Guide

RelaLeap technical implementation note

2026-07-03

Abstract

This document is a practical implementation guide for coding agents who need to implement finite-sample singular learning theory (SLT) estimators for RelaLeap from scratch. It specifies how to implement stochastic gradient Langevin dynamics (SGLD), widely applicable Bayesian information criterion (WBIC) estimation, local learning coefficient (LLC) proxies, block-wise interaction information, calibration benchmarks, and diagnostics. It is not a research paper and it does not claim that the resulting numbers are exact real log canonical thresholds (RLCTs). In RelaLeap reports these numbers must be called finite-sample WBIC/SGLD proxies for RLCT/LLC unless an analytic proof is available.

Contents

1 Purpose and scope **2**

2 Notation and mathematical foundations **3**
2.1 Data, loss, energy, and parameters . . . 3
2.2 Tempered local posterior . . . 3
2.3 SGLD update rule . . . 4
2.4 WBIC and Watanabe free energy . . . 4
2.5 LLC and interaction information . . . 5

3 SGLD sampler implementation **5**
3.1 Required inputs . . . 5
3.2 Pseudocode . . . 5
3.3 Initialization . . . 6
3.4 Step size tuning . . . 7
3.5 Heterogeneous parameter blocks . . . 7
3.6 Burn-in and thinning . . . 8

4 Effective sample size **8**
4.1 Autocorrelation and ESS . . . 8
4.2 Meaning of ESS at least 200 . . . 8

5 WBIC estimation protocol **9**
5.1 Concrete recipe . . . 9
5.2 Temperature sensitivity . . . 9
5.3 Convergence expectations . . . 10

6 LLC interaction information **10**
6.1 Single block, joint block, and interaction . . . 10
6.2 Block-wise versus full-adapter sampling . . . 10

7 Calibration benchmarks **11**
7.1 Benchmark 1: regular identifiable linear regression . . . 11
7.2 Benchmark 2: rank-deficient linear regression . . . 11
7.3 Benchmark 3: overparameterized mixture . . . 11
7.4 Benchmark 4: independent composition . . . 12
7.5 Benchmark 5: redundant composition . . . 12
7.6 Benchmark 6: synergistic composition . . . 12
7.7 Calibration suite pass rule . . . 12

8 Diagnostics and failure modes **13**
8.1 Energy trend: Mann–Kendall test . . . 13
8.2 Divergence and NaN handling . . . 13
8.3 Chains disagree or high CV . . . 13
8.4 Calibration fails . . . 13
8.5 Common pitfalls . . . 14

9 Concrete implementation plan **14**
9.1 Suggested modules . . . 14
9.2 Estimated size . . . 14
9.3 Compute budget . . . 15
9.4 Testing strategy . . . 15

10 Finite-sample caveats **16**

11 Minimum report schema **16**

12 Implementation checklist **17**

1 Purpose and scope

RelaLeap needs SLT estimators because its residual-column learner is meant to find modular residual mechanisms, not merely low reconstruction loss. The implementation must answer questions such as:

(1) Is a trained residual adapter locally simple in the Bayesian evidence sense?

(2) Are two columns approximately independent, redundant, or synergistic?

(3) Do the SLT diagnostics distinguish known calibration regimes before being trusted on real RelaLeap regimes?

The estimators described here must operate over the actual trainable parameter blocks of a RelaLeap residual adapter: atom directions, output directions, amplitudes, gates, routers, memberships, shared cores, and declared joint blocks. A sampler over frozen output amplitudes only is a smoke-test proxy and must not be promoted as SLT evidence.

Core rule. The output of this package is a calibrated diagnostic panel. It is not a certificate that a discovered column is causal. Promotion still requires causal audits, null controls, bootstrap uncertainty, and finite-sample caveats.

2 Notation and mathematical foundations

2.1 Data, loss, energy, and parameters

Let

- $D_n = \{(x_i, y_i)\}_{i=1}^n$ be the estimation set;
- $\theta \in \Theta \subseteq \mathbb{R}^p$ be a chosen parameter block or union of blocks;
- $\hat{\theta}$ be the trained solution, usually the MAP or final trained adapter parameters;
- $\ell_i(\theta)$ be the per-example negative log-likelihood, cross entropy, or residual Gaussian NLL;
- $L_n(\theta) = n^{-1} \sum_{i=1}^n \ell_i(\theta)$ be the mean loss;
- $E_n(\theta) = nL_n(\theta) = \sum_{i=1}^n \ell_i(\theta)$ be the total energy;
- $U(\theta) = -\log \pi(\theta)$ be the negative log prior.

For RelaLeap residual adapters, θ is not the frozen base model. The base transformer and frozen tail may be used to evaluate the loss, but the sampled parameter vector should be the live adapter block under study.

2.2 Tempered local posterior

The local tempered target distribution is

$$p_\beta(\theta \mid D_n) \propto \exp\{-\beta E_n(\theta) - U(\theta)\} \mathbf{1}\{\theta \in \mathcal{N}(\hat{\theta})\}, \tag{1}$$

where $\mathcal{N}(\hat{\theta})$ is the local basin being probed. In code this basin is usually enforced softly by initialization near $\hat{\theta}$, priors, divergence checks, and optional trust-region rejection, not by a hard indicator.

Temperature convention. Watanabe's WBIC uses inverse temperature

$$\beta_{\text{WBIC}} = \frac{1}{\log n} \tag{2}$$

when the exponent multiplies the total energy $E_n = nL_n$. Some RelaLeap checklists store a per-example or API-level temperature as

$$\beta_{\text{api}} = \frac{1}{n \log n}. \tag{3}$$

This is only equivalent if the sampler's internal energy is scaled consistently. Therefore every implementation must record both:

$$\texttt{energy\_definition} \in \{\texttt{mean\_loss}, \texttt{total\_nll}\}, \qquad \texttt{coefficient\_on\_total\_nll}. \tag{4}$$

The intended WBIC target is always

$$p(\theta) \propto \exp\left\{-\frac{E_n(\theta)}{\log n} - U(\theta)\right\}. \tag{5}$$

If a code path says $\beta = 1/(n \log n)$ while also multiplying by total NLL, it is too hot by a factor of n and must fail validation. If a project report requires the checklist field $\beta = 1/(n \log n)$, store it as the per-example convention and also report the effective coefficient on E_n.

2.3 SGLD update rule

For target density $p(\theta) \propto \exp\{-\Phi(\theta)\}$ with

$$\Phi(\theta) = \beta E_n(\theta) + U(\theta), \tag{6}$$

SGLD uses

$$\theta_{t+1} = \theta_t - \frac{\eta_t}{2}\nabla_\theta \Phi(\theta_t) + \sqrt{\eta_t}\,\xi_t, \qquad \xi_t \sim \mathcal{N}(0, I). \tag{7}$$

With minibatches B_t of size m, estimate

$$\nabla E_n(\theta) = \sum_{i=1}^{n} \nabla \ell_i(\theta) \approx \frac{n}{m} \sum_{i \in B_t} \nabla \ell_i(\theta). \tag{8}$$

The implemented update is therefore

$$\theta_{t+1} = \theta_t - \frac{\eta_t}{2}\left[\beta \frac{n}{m} \sum_{i \in B_t} \nabla \ell_i(\theta_t) + \nabla U(\theta_t)\right] + \sqrt{\eta_t}\,\xi_t. \tag{9}$$

2.4 WBIC and Watanabe free energy

In singular models the usual BIC penalty $p \log n/2$ is wrong. Watanabe's asymptotic expansion for Bayesian free energy has the form

$$F_n = nL_n(w_0) + \lambda \log n - (m-1) \log\log n + O_p(1), \tag{10}$$

where λ is the RLCT and m is its multiplicity. For regular identifiable models, $\lambda = p/2$. For singular models, λ may be smaller, and it reflects the local geometry of the loss landscape rather than parameter count.

WBIC estimates the free energy using a tempered posterior at inverse temperature $1/\log n$:

$$\mathrm{WBIC}(n) = \mathbb{E}_{\theta \sim p_{1/\log n}}[E_n(\theta)]. \tag{11}$$

A local finite-sample LLC/RLCT proxy is then

$$\hat{\lambda} = \frac{\mathbb{E}_{\beta_{\mathrm{WBIC}}}[E_n(\theta)] - E_n(\hat{\theta})}{\log n}. \tag{12}$$

This $\hat{\lambda}$ is not "the RLCT." It is a finite-sample proxy whose bias depends on n, prior, local basin, optimization quality, temperature convention, sampler mixing, and omitted $-(m-1)\log\log n$ terms.

2.5 LLC and interaction information

For a block A, estimate $\hat{\lambda}_A$ by sampling only block A while holding the rest of the adapter fixed at trained values, unless a full-adapter conditioning protocol is explicitly registered. For a joint block $A \cup B$, sample the union. Define the LLC interaction information proxy

$$I_\lambda(A; B) = \hat{\lambda}_A + \hat{\lambda}_B - \hat{\lambda}_{A \cup B}. \tag{13}$$

Interpretation:

- $I_\lambda(A; B) \approx 0$: approximately additive or independent local evidence;
- $I_\lambda(A; B) > 0$: redundancy or shared singular structure, because the joint block is simpler than the sum;
- $I_\lambda(A; B) < 0$: synergy or emergent joint structure, because the joint block is more complex than separate blocks.

Only trust this sign after calibration, null normalization, and uncertainty checks.

3 SGLD sampler implementation

3.1 Required inputs

The SGLD module should accept:

(1) a PyTorch module or functional loss closure;

(2) a parameter mask or named list of sampled parameters;

(3) frozen parameter values for all non-sampled blocks;

(4) dataset or minibatch iterator;

(5) prior specification $U(\theta)$;

(6) trained reference $\hat{\theta}$;

(7) temperature convention and effective coefficient on total NLL;

(8) chain settings: seed, steps, burn-in, thinning, initial step size, schedule, divergence thresholds.

3.2 Pseudocode

```
def run_sgld(loss_fn, params, theta_hat, data, prior, cfg):
    set_seed(cfg.seed)
    theta = initialize(theta_hat, cfg.initialization)
    samples = []
    energy_trace = []
    accept_trace = []  # for optional MALA/trust-region rejection

    for t in range(cfg.num_steps):
        batch = next_minibatch(data)
```

```
        loss_mean = loss_fn(theta, batch)       # mean over batch
        grad_loss = grad(loss_mean, params)

        # Unbiased total-energy gradient.
        grad_total = cfg.n_data * grad_loss

        grad_phi = cfg.beta_total_nll * grad_total + prior.grad(theta)

        eta = step_size(t, cfg)
        noise = normal_like(theta, std=sqrt(eta))
        proposal = theta - 0.5 * eta * grad_phi + noise

        if cfg.use_trust_region:
            if is_divergent(proposal) or too_far(proposal, theta_hat, cfg):
                theta = shrink_toward(theta, theta_hat, cfg)
                accept = 0
            else:
                theta = proposal
                accept = 1
        else:
            theta = proposal
            accept = 1

        if t % cfg.energy_eval_interval == 0:
            e = total_energy(loss_fn, theta, full_data_or_fixed_probe)
            energy_trace.append(e)
            check_nan_inf_and_jumps(e, theta)

        if t >= cfg.burn_in and ((t - cfg.burn_in) % cfg.thin == 0):
            samples.append(copy_sampled_params(theta))

        adapt_step_size_if_in_adaptation_window()

    return ChainResult(samples, energy_trace, accept_trace, diagnostics)
```

3.3 Initialization

Use the trained MAP/final adapter state as the default initialization. WBIC/LLC is a local evidence diagnostic around a fitted solution. Starting from random parameters answers a different question and often leaves the relevant basin.

Recommended initializations:

- Default: $\theta_0 = \hat{\theta} + \sigma_{\text{init}}\epsilon$, with small σ_{init} such as 10^{-4} to 10^{-2} times parameter RMS.
- Multi-chain robustness: four or more seeds with independent perturbations around $\hat{\theta}$.
- Random initialization: only for a separate global-basin sensitivity experiment, not the main WBIC estimate.

3.4 Step size tuning

Plain SGLD has no exact accept/reject step, so "acceptance rate" should be interpreted as either an optional MALA-style acceptance rate or a trust-region stability rate. RelaLeap diagnostics may still report an acceptance/stability rate with target range 0.5–0.9.

Practical schedule:

$$\eta_t = \eta_0(1 + t/t_0)^{-\gamma}, \qquad \gamma \in [0.5, 0.75]. \tag{14}$$

For finite-temperature sampling, a small constant step after adaptation is often more useful than a rapidly vanishing step. A recommended implementation is:

(1) adaptation phase: tune η_0 for stability rate 0.5–0.9 and no NaNs;

(2) burn-in phase: fixed or slowly decaying step;

(3) sampling phase: fixed small step, record actual value.

If the chain diverges:

(a) halve the step size and restart the chain from $\hat{\theta}$;

(b) strengthen the local prior or trust-region radius;

(c) check whether the loss is using total instead of mean scaling twice;

(d) check for unconstrained gates, logits, or amplitudes with extreme curvature;

(e) split heterogeneous parameter blocks and tune separate preconditioners.

3.5 Heterogeneous parameter blocks

RelaLeap adapters have heterogeneous geometry:

- atom input directions a_g and output directions b_g may live naturally on normalized spheres;
- amplitudes β_g are scalar and often sharp near zero;
- gates and router logits can have saturation and scale symmetries;
- memberships q_{kg} may live on a simplex;
- shared-core parameters can create singular ridges.

Do not use one global step size blindly. Implement block-wise preconditioning:

$$\theta_{t+1}^{(b)} = \theta_t^{(b)} - \frac{\eta_t^{(b)}}{2} P_b \nabla_b \Phi(\theta_t) + \sqrt{\eta_t^{(b)}}\, P_b^{1/2} \xi_t^{(b)}, \tag{15}$$

where P_b may be diagonal RMS, Adam-style second moment frozen during sampling, Fisher diagonal, or a simple scale based on parameter RMS. Record all preconditioners.

For constrained blocks, prefer unconstrained internal parameters plus transforms:

- normalized directions: sample raw vector v, use $a = v/(\|v\| + \epsilon)$;
- positive scales: sample ρ, use $\sigma = \mathrm{softplus}(\rho)$;
- simplex memberships: sample logits and apply softmax or sparsemax.

Include the log-Jacobian in the prior only if the target density is meant to be over the constrained parameter. For a first implementation, document the unconstrained prior and treat transformed geometry as part of the model parameterization.

3.6 Burn-in and thinning

Minimum RelaLeap synthetic settings:

- at least 4 chains;
- at least 10,000 steps per chain for small CPU benchmarks;
- burn-in at least 20 percent;
- thinning only for storage, never as a substitute for ESS.

For larger adapters, 50,000 or more steps may be required. Save post-burn-in energy traces even when parameter samples are thinned.

4 Effective sample size

4.1 Autocorrelation and ESS

For each chain, compute ESS for the scalar statistic used in WBIC, usually total energy $E_n(\theta)$. Given post-burn-in values $z_1, \ldots, z_T$, estimate autocorrelation

$$\hat{\rho}_k = \frac{\sum_{t=1}^{T-k}(z_t - \bar{z})(z_{t+k} - \bar{z})}{\sum_{t=1}^{T}(z_t - \bar{z})^2}. \tag{16}$$

The integrated autocorrelation time is

$$\hat{\tau} = 1 + 2\sum_{k=1}^{K} \hat{\rho}_k, \tag{17}$$

where K is chosen by an initial-positive-sequence or initial-monotone-sequence rule. Then

$$\mathrm{ESS} = \frac{T}{\hat{\tau}}. \tag{18}$$

Also compute split-chain $\hat{R}$ for energy when possible. Energy ESS is the mandatory gate; parameter-wise ESS is useful but not sufficient.

4.2 Meaning of ESS at least 200

ESS ≥ 200 per chain for energy means the Monte Carlo mean of $E_n(\theta)$ is based on roughly 200 independent energy draws after accounting for autocorrelation. It does not guarantee correct basin coverage or unbiased RLCT estimation. It only says that the chain's scalar energy average is not obviously dominated by serial correlation.

If ESS is too low:

(1) run longer chains;

(2) reduce step size if traces show jumps or instability;

(3) increase step size cautiously if traces are sticky and stable;

(4) improve preconditioning by parameter block;

(5) reparameterize saturated gates or normalized directions;

(6) split a joint block into smaller diagnostic blocks;

(7) compare SGLD with NUTS/HMC on tiny calibration models if available.

If ESS remains below 200, mark the estimate diagnostic only and set `slt_evidence_status = uncalibrated`.

5 WBIC estimation protocol

5.1 Concrete recipe

For each model, block, data split, and temperature:

(1) Train or load the adapter and record $\hat{\theta}$.

(2) Freeze non-sampled blocks at trained values.

(3) Compute $E_n(\hat{\theta})$ on the estimation set.

(4) Run at least 4 independent SGLD chains at WBIC temperature.

(5) Discard burn-in, compute energy samples $E_n(\theta_s)$ on a fixed full-data or deterministic probe set.

(6) Estimate

$$\widehat{\mathrm{WBIC}} = \bar{E} = \frac{1}{S}\sum_{s=1}^{S} E_n(\theta_s). \tag{19}$$

(7) Estimate

$$\hat{\lambda} = \frac{\bar{E} - E_n(\hat{\theta})}{\log n}. \tag{20}$$

(8) Report chain-wise $\hat{\lambda}_c$, pooled $\hat{\lambda}$, standard error, bootstrap confidence interval, ESS, trend tests, divergence counts, and coefficient of variation across seeds.

Uncertainty should include chain variability and Monte Carlo autocorrelation. A simple first implementation can report:

$$\mathrm{SE}(\hat{\lambda}) = \frac{\mathrm{SE}_{\mathrm{chains}}(\bar{E}_c)}{\log n}, \tag{21}$$

plus a block bootstrap over post-burn-in energy traces. For final gates, also resample input examples.

5.2 Temperature sensitivity

Run at three effective total-energy coefficients:

$$\beta/4, \qquad \beta, \qquad 4\beta, \tag{22}$$

where $\beta = 1/\log n$ in the total-energy convention. If the checklist API convention is used, the recorded per-example temperatures are $1/(4n\log n)$, $1/(n\log n)$, and $4/(n\log n)$, but the effective total-energy target must still be clear.

The central estimate should be finite and stable. The energy mean should change monotonically in the sensible direction: colder posterior (larger inverse temperature) should concentrate nearer $\hat{\theta}$ and typically reduce sampled energy; hotter posterior should explore higher energy. Wild nonmonotonicity, divergence at the central temperature, or disagreement of signs in I_λ means diagnostic only.

5.3 Convergence expectations

A converged WBIC run usually has:

- post-burn-in energy trace without significant monotone trend;
- no NaN or Inf samples;
- no single-step post-burn-in energy jumps above 5 empirical standard deviations unless explained and filtered;
- ESS ≥ 200 per chain for energy;
- seed CV of $\hat{\lambda}$ below 0.25;
- central temperature estimate bracketed by sensitivity runs;
- agreement with calibration benchmark expectations.

6 LLC interaction information

6.1 Single block, joint block, and interaction

For each declared pair (A, B):

(1) Estimate $\hat{\lambda}_A$ by sampling only block A.

(2) Estimate $\hat{\lambda}_B$ by sampling only block B.

(3) Estimate $\hat{\lambda}_{AB}$ by sampling the union $A \cup B$.

(4) Use identical data split, loss, prior family, temperature convention, chain count, burn-in, ESS gate, and input batches.

(5) Compute $I_\lambda(A; B) = \hat{\lambda}_A + \hat{\lambda}_B - \hat{\lambda}_{AB}$.

(6) Compute uncertainty by pairing bootstrap samples of the three estimates.

6.2 Block-wise versus full-adapter sampling

Block-wise sampling. Cheap, interpretable, and appropriate for calibration and routine audits. It can miss singularities that require other blocks to move jointly.

Full-adapter sampling with projections. More faithful to global local evidence, but expensive and harder to attribute. Use for at least one full-adapter run after block-wise estimates pass diagnostics.

Recommended RelaLeap policy. Run block-wise estimates for all columns and declared pairs, plus one full-adapter estimate. If block-wise I_λ suggests sparse structure but full-adapter sampling shows diffuse all-to-all interaction, the columnar interpretation is blocked.

7 Calibration benchmarks

All benchmarks should be CPU-sized: $n \in \{256, 512, 1024\}$, dimensions 2–16, four chains, 10k steps per chain initially. Use Gaussian NLL with known noise variance unless otherwise specified.

7.1 Benchmark 1: regular identifiable linear regression

Specification:

$$x_i \sim \mathcal{N}(0, I_d), \qquad y_i = x_i^T w^\star + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma^2), \tag{23}$$

with parameter $w \in \mathbb{R}^d$, $d = 4$ or 8.

Known behavior: regular identifiable model with RLCT $\lambda = d/2$.

Pass criterion: $\hat{\lambda}$ within about $\pm 20\%$ of $d/2$, stable across seeds, ESS pass.

Why: verifies scaling, temperature convention, Gaussian prior, and WBIC formula in the easiest case.

7.2 Benchmark 2: rank-deficient linear regression

Specification:

$$y_i = z_i^T u^\star + \epsilon_i, \qquad x_i = R z_i, \tag{24}$$

where $z_i \in \mathbb{R}^r$, $x_i \in \mathbb{R}^d$, $R \in \mathbb{R}^{d \times r}$ has rank $r < d$, and the fitted parameter is $w \in \mathbb{R}^d$ with prediction $x_i^T w$. Use $d = 8$, $r = 3$.

Expected behavior: only r identifiable directions affect predictions, so the effective regular contribution is approximately $r/2$; null directions should not contribute like full parameters.

Pass criterion: $\hat{\lambda}$ closer to $r/2$ than to $d/2$, within rough tolerance. Report caveat that priors and finite ridge width can perturb the estimate.

Why: catches implementations that merely return parameter count over two.

7.3 Benchmark 3: overparameterized mixture

Specification: one-dimensional two-component Gaussian mixture fitted to data generated by one component:

$$p(y \mid \theta) = \alpha \mathcal{N}(y; \mu_1, \sigma^2) + (1 - \alpha)\mathcal{N}(y; \mu_2, \sigma^2), \tag{25}$$

with data $y_i \sim \mathcal{N}(0, \sigma^2)$, parameters $(\operatorname{logit} \alpha, \mu_1, \mu_2)$, and fixed σ.

Expected behavior: singular due to permutation symmetry and component redundancy; $\lambda < p/2$ where $p = 3$.

Pass criterion: estimator detects singularity by producing $\hat{\lambda}$ clearly below 1.5 and stable enough for sign/magnitude classification.

Why: verifies that the estimator can see a classic singular model.

7.4 Benchmark 4: independent composition

Specification:

$$y_i = f_A(x_{iA}; \theta_A) + f_B(x_{iB}; \theta_B) + \epsilon_i, \tag{26}$$

where x_{iA} and x_{iB} are independent, f_A and f_B are small regular linear or one-hidden-unit modules with disjoint parameters.

Expected behavior:

$$\lambda_{A\cup B} \approx \lambda_A + \lambda_B, \qquad I_\lambda(A; B) \approx 0. \tag{27}$$

Pass criterion: additivity within about 15% and confidence interval for I_λ includes zero.

Why: catches false interaction caused by inconsistent splits, priors, or sampler settings.

7.5 Benchmark 5: redundant composition

Specification:

$$y_i = c\,[a(x_i) + b(x_i)] + \epsilon_i \tag{28}$$

or a shared-core residual model where two columns use the same latent scalar c or same direction u:

$$r_A(h) = \alpha_A u\, s(h), \qquad r_B(h) = \alpha_B u\, s(h). \tag{29}$$

Sample blocks $A = (\alpha_A)$, $B = (\alpha_B)$, and joint $AB = (\alpha_A, \alpha_B)$ or include the shared core in a declared shared block.

Expected behavior:

$$\lambda_{A\cup B} < \lambda_A + \lambda_B, \qquad I_\lambda(A; B) > 0. \tag{30}$$

Pass criterion: positive I_λ with CI mostly above zero.

Why: verifies that shared singular structure appears as redundancy.

7.6 Benchmark 6: synergistic composition

Specification: target depends on a product or gated pair that cannot be represented by either block alone:

$$y_i = \theta_A \theta_B\, s(x_i) + \epsilon_i, \tag{31}$$

with trained solution near nonzero product, or a residual pair where a gate in A activates a direction in B. A more RelaLeap-like version is

$$r(h, c) = m_A(c) m_B(c)\, v\, \phi(u^T h). \tag{32}$$

Expected behavior: joint movement reveals structure not present in separate block probes. Depending on exact generator and conditioning, $I_\lambda(A; B) < 0$ or at least significantly nonzero in the preregistered synergistic direction.

Pass criterion: correct sign under the chosen generator and a CI excluding the independent near-zero band.

Why: verifies that the estimator does not force additivity and can flag emergent joint structure.

7.7 Calibration suite pass rule

At least 5 of 6 benchmarks must show correct sign and rough magnitude. Regular and independent benchmarks are mandatory. If fewer than 5 pass, no RelaLeap LLC/WBIC number may be treated as calibrated evidence.

8 Diagnostics and failure modes

8.1 Energy trend: Mann–Kendall test

For each post-burn-in energy trace, run a Mann–Kendall monotone trend test. Requirement: $p > 0.05$ for no significant trend. If the trend test fails, the chain is still warming up, drifting between basins, or numerically unstable. Increase burn-in, reduce step size, strengthen local prior, or split blocks.

8.2 Divergence and NaN handling

Flag divergence if any of the following occurs:

- NaN or Inf loss, gradient, parameter, or energy;
- post-burn-in energy jump greater than 5 standard deviations;
- parameter norm exceeds a configured multiple of trained norm;
- trust-region rejection/stability rate outside 0.5–0.9;
- loss exceeds a hard ceiling based on baseline loss.

Do not silently drop bad samples. Count them, mark the chain failed if they recur, and restart with smaller step size. If any central-temperature chain repeatedly fails, mark the estimate uncalibrated.

8.3 Chains disagree or high CV

If chain-wise $\hat{\lambda}$ has coefficient of variation ≥ 0.25:

(1) inspect whether chains occupy different basins;

(2) rerun with smaller initialization perturbation;

(3) increase chain length and burn-in;

(4) check exact temperature scaling;

(5) compare block-wise and full-block curvature;

(6) report diagnostic only if disagreement persists.

8.4 Calibration fails

When calibration fails, do not tune on RelaLeap outcomes. Fix in this order:

(1) regular linear benchmark: temperature/loss scaling bug likely;

(2) rank-deficient benchmark: prior, ridge, or block mask bug likely;

(3) mixture benchmark: sampler not exploring singular ridge or label symmetry;

(4) independent composition: inconsistent data, prior, or temperature across block and joint runs;

(5) redundant/synergistic composition: block definition or interpretation mismatch.

8.5 Common pitfalls

- using $1/(n \log n)$ on total NLL and accidentally sampling a much hotter target;
- sampling frozen scalar knobs instead of actual adapter parameters;
- forgetting prior gradients;
- including frozen base model parameters in the sampled block;
- using train mini-batch noise but evaluating E_n on changing batches;
- interpreting low ESS as success because many raw samples were saved;
- comparing λ_A, λ_B, and λ_{AB} with different priors or data splits;
- forcing all interactions toward zero and thereby hiding true synergy;
- reporting exact RLCT language for finite-sample proxies.

9 Concrete implementation plan

9.1 Suggested modules

```
relaleap/slt/__init__.py
relaleap/slt/parameter_blocks.py      # block discovery, masks, flatten/unflatten
relaleap/slt/priors.py                # Gaussian, scale-aware, transformed priors
relaleap/slt/sgld.py                  # SGLD kernel, schedules, diagnostics
relaleap/slt/wbic.py                  # WBIC orchestration and lambda estimates
relaleap/slt/ess.py                   # autocorrelation, ESS, R-hat
relaleap/slt/trend.py                 # Mann-Kendall and jump tests
relaleap/slt/llc_interactions.py      # block, joint, I_lambda matrix
relaleap/slt/calibration_benchmarks.py# six benchmark generators
relaleap/slt/reports.py               # JSON/CSV/Markdown report writers
tests/test_slt_scaling.py
tests/test_slt_calibration.py
tests/test_llc_interactions.py
```

9.2 Estimated size

A clean first implementation should be about:

- parameter block utilities: 200–350 lines;
- priors and transforms: 150–300 lines;
- SGLD kernel: 300–500 lines;
- diagnostics ESS/trend/R-hat: 250–400 lines;
- WBIC orchestration: 250–450 lines;
- LLC interaction orchestration: 200–350 lines;
- calibration benchmarks: 500–900 lines;
- tests and fixtures: 500–1000 lines.

9.3 Compute budget

No GPU or paid compute is needed for calibration. CPU target:

- one small benchmark: 4 chains $\times$ 10k steps in minutes;
- full six-benchmark suite: under about one hour for small dimensions;
- RelaLeap synthetic block-wise audit: minutes to tens of minutes depending on adapter size;
- real transformer residual caches: only after synthetic gates pass.

9.4 Testing strategy

Unit tests:

- flatten/unflatten round trip preserves parameters and gradients;
- block masks update only selected parameters;
- SGLD noise variance equals step size;
- prior gradients match finite differences;
- ESS returns near T for iid noise and much smaller values for AR(1) traces;
- temperature convention test verifies effective coefficient on total NLL.

Integration tests:

- regular linear $\hat{\lambda} \approx d/2$;
- rank-deficient estimate closer to $r/2$ than $d/2$;
- independent composition $I_\lambda \approx 0$;
- redundant composition $I_\lambda > 0$;
- synergistic composition correct sign;
- NaN injection causes chain failure, not silent success.

Every run should write:

```
runs/slt/<run_id>/config.yaml
runs/slt/<run_id>/summary.json
runs/slt/<run_id>/chain_diagnostics.csv
runs/slt/<run_id>/energy_traces.npy
runs/slt/<run_id>/lambda_estimates.csv
runs/slt/<run_id>/decision.md
```

10 Finite-sample caveats

The estimator can support statements such as:

> At this sample size, with this prior, block definition, temperature convention, sampler, and calibration suite, the finite-sample WBIC/SGLD proxy suggests lower local evidence complexity for model A than model B.

It cannot by itself prove:

- the exact RLCT of a neural adapter;
- that a learned residual column is causal;
- that an interaction sign is stable outside the sampled basin;
- that a finite-data WBIC proxy equals asymptotic local free energy;
- that a failed estimate means no singular structure exists.

Trust the estimates only when:

(1) calibration passes;

(2) ESS, trend, divergence, and seed CV gates pass;

(3) temperature sensitivity is sensible;

(4) matched null controls are beaten with uncertainty;

(5) sample-size sensitivity is stable;

(6) conclusions use finite-sample proxy wording.

If any gate fails, report `slt_evidence_status = uncalibrated` and fail closed for scientific promotion.

11 Minimum report schema

Each RelaLeap SLT report must include:

```
estimator_type: WBIC_SGLD
estimator_status: finite_sample_wbic_proxy_not_exact_rlct
energy_definition: total_nll | mean_loss
coefficient_on_total_nll: <float>
checklist_beta_per_example: <float or null>
n_data: <int>
parameter_blocks: [names]
chain_count: <int>
steps_per_chain: <int>
burn_in: <int>
thin: <int>
ess_energy_per_chain: [floats]
```

```
mann_kendall_p_per_chain: [floats]
divergence_or_nan_count: <int>
seed_lambda_values: [floats]
seed_cv: <float>
temperature_sensitivity: [beta_over_4, beta, four_beta]
lambda_hat: <float>
lambda_ci95: [lo, hi]
calibration_benchmarks_passed: <int>/6
null_normalized_margin: <float>
finite_sample_caveat: "These are finite-sample WBIC/SGLD proxies, not exact RLCTs."
```

12 Implementation checklist

Before a coding agent marks the estimator complete:

(1) regular linear benchmark passes;

(2) all six calibration generators exist and run on CPU;

(3) SGLD samples actual live parameter blocks;

(4) WBIC records effective coefficient on total NLL and avoids temperature ambiguity;

(5) ESS, Mann–Kendall, divergence, NaN, jump, and seed-CV diagnostics are automatic;

(6) I_λ uses matched block, joint, data, prior, and temperature settings;

(7) failed diagnostics automatically downgrade evidence status;

(8) reports use finite-sample proxy language.